

Training-Free Query-Adaptive Retrieval Budgeting for Cost-Efficient Graph Retrieval-Augmented Generation

Anonymous Author(s)

ABSTRACT

Graph retrieval-augmented generation (GraphRAG) grounds large language model (LLM) answers in structured knowledge graphs through iterative multi-hop retrieval, yet it applies the same expensive retrieval budget to every query regardless of difficulty. Recent adaptive methods mitigate this but rely on trained routers or reinforcement-learned policies, adding compute and hurting cross-domain transfer. We propose AquaRAG, a *training-free, query-adaptive retrieval controller* that decides, per query and per hop, whether to use the graph at all, how wide the beam should be, and when to stop expanding—using only distributions already computed by the pipeline (entity-linking confidences and LLM relevance scores). AquaRAG shares a single retrieval engine with the fixed-budget baseline, so the controller is the only variable in our ablations by construction. On MetaQA with a 7B generator, AquaRAG improves F1 by 10.3% (0.343→0.378) and Hit@1 by 7.0% at 2-hop reasoning while reducing retrieval cost by 23.2% (1111→853 input tokens/query), a gap that is statistically significant ($p=0.049$, paired bootstrap). On mixed-depth query streams—where no single fixed budget is correct—AquaRAG outperforms both a logistic-regression and an MLP trained router (up to +14.7% F1) without any training data, and matches the fixed baseline’s accuracy at 24% lower cost. The cost saving transfers to a second KG (WebQSP/Freebase, −19%), though F1 trades slightly lower on this denser graph. The advantage grows with reasoning depth: at 2-hop with a 1.5B model, F1 improves 23.5% ($p=0.004$) at 19% lower cost. Ablations isolate each controller component—early stopping drives essentially all of the cost saving under perfect linking—and a noise-corrupted linking study shows the use-graph decision, when isolated, is a controllable cost–accuracy trade-off (it cuts cost 10.5% at a small accuracy cost). We report honest boundary conditions: on trivial single-hop queries the controller trades a small accuracy loss for cost savings, and on weak-judge high-depth settings graph retrieval itself becomes harmful. AquaRAG requires no training, no learned parameters, and runs on a single 32 GB GPU.

1 INTRODUCTION

Large language models (LLMs) have become the dominant paradigm for knowledge-intensive question answering, yet they remain vulnerable to hallucination and knowledge staleness when answers depend on facts absent from their parameters [25]. Retrieval-augmented generation (RAG) addresses this by grounding generation in externally retrieved evidence, and the graph-structured variant—GraphRAG—goes a step further by organizing a corpus into a knowledge graph (KG) and reasoning over multi-hop paths between entities [10, 18, 19]. Multi-hop GraphRAG has proven especially valuable for compositional questions whose answers require chaining relations across several hops, a setting where flat vector retrieval fundamentally fails [11]. The canonical GraphRAG retrieval

loop, exemplified by Think-on-Graph (ToG) [18] and its successors [7, 8, 19], iteratively expands a frontier of candidate triples, invokes an LLM as a relevance judge to score each candidate, prunes to a fixed beam, and repeats for a predetermined number of hops.

This fixed-budget retrieval policy, however, ignores a fundamental property of real query streams: *questions vary enormously in reasoning difficulty*. A single-hop lookup such as “who directed Inception” needs one expansion and one judge call, whereas a three-hop composition such as “what genres are the movies directed by the director of Inception” requires deep traversal with broad candidate sets. Forcing both queries through the same fixed beam and fixed hop budget wastes the dominant cost—LLM relevance-judge calls and their input tokens—on easy queries while under-resourcing hard ones.

A motivating example. A trivial lookup (“what year was Titanic released”) is answered after one hop, yet a fixed 3-hop policy runs two more judge calls at $\sim 2\times$ the necessary cost; a hard 3-hop query instead needs a wider beam at an intermediate hop that a fixed beam prunes. In our experiments AquaRAG stops after 2 hops for 55% of 2-hop queries with no accuracy loss, so the third hop is frequently uninformative—a signal existing fixed-budget controllers compute but ignore. A well-tuned fixed budget is nonetheless a strong baseline, so we compare against tuned Fixed- K and beam-swept variants (Section 4.3) and, crucially, on mixed-depth streams (Section 4.4) where no single budget is correct.

Recent work has begun to address this heterogeneity. *Use-Graph-When-It-Needs* [1] trains a detector to decide whether a query needs the graph, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over GraphRAG backends. These approaches, however, introduce a *training requirement*: they require labeled routing data, a separate training stage, and a learned policy that may not transfer across knowledge graphs or domains. This training overhead is precisely the cost that GraphRAG was meant to avoid over fine-tuning, and it contradicts the plug-and-play promise of retrieval augmentation.

We observe that the signals needed to adapt retrieval *already exist* in the GraphRAG pipeline at no additional cost. Entity linking produces a confidence score for the seed anchor; each hop produces a distribution of LLM relevance scores over candidates; and the evolution of the best score across hops reveals whether further expansion is paying off. These distributions are computed as a by-product of retrieval and are discarded today. We argue that a controller operating purely on these distributions—with no learned parameters, no training data, and no gradient updates—can capture much of the difficulty-adaptivity that trained routers target, without their training cost, and applies unchanged wherever entity linking and relevance judging are defined. The key insight is that retrieval difficulty is observable as a property of the *query–pipeline interaction*, not of the query alone, and this interaction is exposed through the score distributions the pipeline already produces.

We propose AquaRAG (Adaptive Query-budgeted RAG), a training-free, query-adaptive retrieval controller for GraphRAG. AquaRAG makes three per-query, per-hop decisions using only distributional signals: (i) a *use-graph decision* that routes low-confidence-anchored queries to a cheap graph-free answer path; (ii) an *adaptive beam allocator* that widens the beam when hop-level relevance scores are flat (uncertain) and narrows it when one candidate dominates; and (iii) a *marginal-gain early-stopping rule* that halts expansion once the best per-hop relevance score stops improving, for which we give an accounting of its guaranteed cost reduction. Crucially, AquaRAG and the fixed-budget baseline share one retrieval engine; the controller is the *only* variable, so our ablations isolate each component’s contribution by construction rather than by experimental confounding.

We evaluate AquaRAG on MetaQA [24], the standard multi-hop KGQA benchmark used by ToG and GNN-RAG, across 1/2/3-hop reasoning depths, two generator scales (1.5B and 7B parameters), a mixed-depth query stream, and multiple seeds. Our headline result is that on 2-hop reasoning with a 7B generator—the setting that best balances difficulty and judge reliability—AquaRAG improves F1, Hit@1, and retrieval cost simultaneously over the fixed baseline (with statistical significance), and remains on or above the cost-accuracy frontier of *tuned* fixed-budget baselines. We additionally report honest boundary conditions where the controller trades accuracy for cost (trivial 1-hop) or where graph retrieval itself is harmful (weak-judge, high-depth), and we show via a noise-corruption study that the use-graph decision becomes a controllable cost lever when entity linking is unreliable.

Contributions.

- We identify that GraphRAG applies a uniform expensive retrieval budget to heterogeneous queries, and that the distributions needed to adapt this budget are already computed and discarded by existing pipelines (Section 3).
- We propose AquaRAG, a training-free controller whose primary mechanism is marginal-gain early stopping, augmented by two conditional levers—adaptive beam (helpful when hop scores are uncertain) and use-graph (active under noisy linking). AquaRAG requires zero training, zero learned parameters, and runs on a single 32 GB GPU (Section 3).
- We show that AquaRAG improves F1, Hit@1, and cost simultaneously over the fixed ToG-style baseline at 2-hop with a 7B generator (F1 +10.3%, cost −23.2%, $p=0.049$), and on mixed-depth streams it outperforms *both* a logistic-regression and an MLP trained router (up to +14.7% F1)—without any training data (Section 4.5). The cost saving transfers to a second KG (WebQSP/Freebase, −19%). Ablations isolate each component’s contribution (Section 4).
- We characterize the method’s boundaries honestly: on single-depth splits a tuned fixed budget or MLP beats AquaRAG, F1 trades lower on denser KGs, and weak judges at depth make graph retrieval harmful (Section 4.9).

2 RELATED WORK

Graph retrieval-augmented generation. GraphRAG extends flat vector RAG by reasoning over a knowledge graph, and iterative

multi-hop retrieval has become the dominant paradigm. Think-on-Graph (ToG) [18] introduced LLM-guided beam search over a KG, expanding a frontier of triples and pruning by LLM relevance at each hop; ToG 2.0 [8] added knowledge-guided retrieval augmentation, and Search-on-Graph [19] refined iterative informed navigation with informed path selection. GNN-RAG [10] decouples graph reasoning from generation by training a GNN to score candidate paths, then feeding the top paths to the LLM, while Paths-over-Graph [20] and Fast-ToG [7] pursue wider or faster reasoning through path enumeration and pruning heuristics. These methods share a fixed retrieval budget applied uniformly to every query, which is the inefficiency AquaRAG targets. Recent surveys [5, 12, 23] confirm the field’s rapid growth and the emerging emphasis on cost-aware retrieval.

Adaptive and cost-efficient GraphRAG. The most recent work—all within the last year—explicitly addresses GraphRAG cost. *Use-Graph-When-It-Needs* [1] learns when to invoke the graph via a trained detector, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over backends of varying cost. Core-based Hierarchies [4] reduce indexing cost via k -core decomposition, EHRAG [16] builds lightweight hypergraph indices, while S-Path-RAG [3] and the minimal-subgraph retriever of Yuan et al. [22] prune the retrieval path itself by selecting shortest or sufficient subgraphs. These approaches either *train* a router/detector (adding a learning stage and a domain-specific policy) or optimize the *index* or *path selection* rather than per-query *retrieval budget* (beam width, hop count, graph-vs-no-graph). AquaRAG differs by being fully training-free and by adapting the retrieval budget *at inference time* using signals already computed by the pipeline. SkewRoute [21] is closest in spirit—it routes using score-skewness—but targets LLM-backend selection (large vs. small model), not graph-retrieval budgeting.

Training-free adaptation and early stopping. The idea of adapting inference compute to input difficulty without training appears in adaptive reasoning-suppression for reasoning LMs [26], token-efficient prompting [14], and early-exit transformers. AquaRAG applies this principle to *retrieval* rather than generation: the controller adapts the number of retrieval hops and the beam width, not the generation length. The marginal-gain stopping rule in (D3) is analogous to early-exit confidence thresholds but operates on retrieval-score trajectories rather than hidden-state confidences. A key distinction is that our signals (link confidence, hop-score entropy) are *causal*—they measure the pipeline’s actual state during retrieval—rather than *predictive* features estimated before retrieval begins, which is why no training is needed.

Knowledge-graph question answering. MetaQA [24] is the canonical multi-hop KGQA benchmark, built on the WikiMovies KG, with earlier neural approaches including EmbedKGQA [13], PullNet [17], ReaRev [9], and TransferNet [15] that learn graph embeddings or attention over relations. The LLM era shifted KGQA from learned graph embeddings to LLM-guided retrieval [6, 10, 18], where the LLM itself scores candidate triples at each hop. We adopt MetaQA for comparability with this line and report F1/Hit@1 under the same protocol as ToG and GNN-RAG, while adding cost metrics

(tokens, calls, hops) that prior work does not report but that are essential for deployment.

3 METHOD

3.1 Preliminaries: GraphRAG Retrieval

We formalize the GraphRAG retrieval loop that our controller augments. Given a knowledge graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ with entities $\mathcal{E}$, relations $\mathcal{R}$, and triples $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$, and a natural-language question q , the goal is to retrieve a reasoning subgraph $\mathcal{S} \subseteq \mathcal{T}$ that, when provided as context to a generator LLM G , yields the correct answer.

The retrieval proceeds iteratively. Let $\mathcal{F}_k \subseteq \mathcal{E}$ denote the *frontier*—entities to expand—at hop k . The pipeline consists of four steps per hop:

(1) **Entity linking** maps the question to seed entities $\mathcal{F}_0 \subseteq \mathcal{E}$, either by exact match (when entity annotations are available) or by embedding nearest-neighbor search. Each linked entity carries a confidence score $\ell_i \in [0, 1]$ (cosine similarity).

(2) **Frontier expansion** produces candidate triples by traversing one edge from each frontier entity: $C_k = \{(e, r, e') \in \mathcal{T} : e \in \mathcal{F}_k, e' \notin \mathcal{V}\}$, where $\mathcal{V}$ is the set of already-visited entities. To bound prompt length, $|C_k|$ is capped at $c_{\max}$.

(3) **Relevance judging** scores each candidate triple via an LLM call: $r_{k,j} = J(\text{triple}_j, q) \in [0, 1]$. This is the *dominant cost*: each call consumes input tokens proportional to $|C_k|$, and there is one such call per hop. The judge prompt batches all candidates into a single call to amortize the fixed overhead of the instruction tokens.

(4) **Pruning and frontier update** retains the top- b_k triples by score as $\mathcal{S}_k$, adds their entities to $\mathcal{V}$, and sets $\mathcal{F}_{k+1}$ to the newly discovered entities. After K hops (or an early stop), the union $\bigcup_k \mathcal{S}_k$ is the reasoning subgraph fed to G .

A *retrieval controller* π governs three quantities per query: whether to use the graph at all ($u_0 \in \{0, 1\}$), the beam width (b_k), and whether to stop ($s_k \in \{0, 1\}$). The fixed-budget baseline (ToG-style) sets $u_0=1$, $b_k=B$ constant, $s_k=0$ until $k=K$. AquaRAG replaces this with a training-free controller that reads distributional signals computed as by-products of steps (1)–(3). Because the retrieval engine (steps 1–4) is identical across controllers, comparing AquaRAG to the baseline isolates the controller as the sole variable.

3.2 Distributional Signals

AquaRAG exploits three signals that the pipeline already computes:

(S1) **Link-confidence vector**. Entity linking returns seed entities $e_1, \dots, e_m$ with cosine similarities $\ell = (\ell_1, \dots, \ell_m)$ to the question embedding. The top value $\ell_{(1)}$ measures how reliably the seed anchor matches the question; a low $\ell_{(1)}$ signals that multi-hop expansion will amplify a wrong seed.

(S2) **Hop relevance distribution**. At hop k , the judge produces scores $r_k = (r_{k,1}, \dots, r_{k,|C_k|})$ over candidates. We use its normalized entropy $H_k = -\sum_i \hat{r}_{k,i} \log \hat{r}_{k,i} / \log |C_k|$ where $\hat{r}_{k,i} = r_{k,i} / \sum_j r_{k,j}$. Low H_k means one candidate dominates (confident hop); high H_k means scores are flat (uncertain hop, many plausible continuations).

(S3) **Per-hop top-score trajectory**. The sequence $t_k = \max_i r_{k,i}$ tracks whether expansion is uncovering more relevant evidence. A plateau or decline in t_k indicates diminishing returns.

3.3 The AquaRAG Controller

AquaRAG makes three training-free decisions from (S1)–(S3).

(D1) *Use-graph decision*. If $\ell_{(1)} < \theta_{\text{low}}$, the seed anchor is too unreliable for multi-hop expansion to help; we route q to a graph-free answer generated directly from the question. This saves all K judge calls for that query. The threshold θ_{low} is the only hyperparameter governing (D1) and, as we show, is inert under perfect linking (it never fires) but decisive under noisy linking.

(D2) *Adaptive beam allocation*. We set the beam at hop k to

$$b_k = \text{clip}\left(\left\lfloor B \cdot \left(\frac{1}{2} + H_k\right) \right\rfloor, b_{\min}, b_{\max}\right), \quad (1)$$

where B is a base beam. The factor $\frac{1}{2} + H_k$ maps entropy $H_k \in [0, 1]$ to a multiplier in $[0.5, 1.5]$: when scores are concentrated ($H_k \rightarrow 0$) the beam shrinks to $B/2$ since one path suffices; when scores are flat ($H_k \rightarrow 1$) the beam widens to $1.5B$ to avoid pruning the true continuation among many equally-scored candidates. This entropy-to-beam map is the simplest monotone allocation consistent with the intuition that uncertainty demands exploration.

(D3) *Marginal-gain early stopping*. We stop after hop $k \geq 1$ when the top-score gain fails to exceed a margin:

$$s_k = 1 \iff (t_k - t_{k-1}) \leq \delta \text{ or } t_k < \tau \text{ or } \mathcal{F}_{k+1} = \emptyset. \quad (2)$$

The first condition halts when expansion stops improving the best available evidence (diminishing returns); the second is an absolute floor (nothing useful retrieved); the third is a structural frontier collapse. The default δ is chosen so that a hop whose top score does not beat the previous hop's top by a meaningful margin—indicating the new frontier is no more relevant than the old—is not worth its judge cost.

Assumptions on the signals, and their limits. Both (D2) and (D3) treat the raw LLM relevance scores as if they were meaningful across candidates and across hops, which requires care. The judge emits integers in $[0, 100]$ that are *not* calibrated probabilities, so the entropy H_k in (D2) is a heuristic dispersion measure, not an information-theoretic quantity; we normalize per hop to reduce scale sensitivity, and when all scores are zero we fall back to the base beam B (the normalization is undefined there). (D3) further assumes the per-hop top score t_k is comparable across hops, which can fail: a necessary later-hop relation may have lower surface relevance than the first hop, so a score decrease does not strictly imply diminishing evidential value. We therefore do not claim these signals are optimal or calibrated; we claim only that they are *informative enough* to drive useful budget decisions, and we validate this empirically—the sensitivity sweep (Figure 3) shows the decisions are robust to the thresholds, and the 1.5B/3-hop boundary case (Section 4.9.5) shows exactly where the assumption breaks: a weak judge produces an unreliable t_k trajectory, so (D3) misfires. A calibrated judge or a learned score-to-decision map is a natural extension we leave to future work.

REMARK 1 (CALL-COUNT ACCOUNTING). Let C_{fix} and C_{ada} be the expected per-query judge-call count of the fixed and AquaRAG controllers under the same engine. If a fraction ρ_u of queries satisfy (D1)'s skip condition and the mean stopping hop under (D3) is $\bar{k}_{\text{stop}} \leq K$, then $C_{\text{ada}} = (1 - \rho_u) \bar{k}_{\text{stop}} + \rho_u K = (1 - \rho_u) C_{\text{fix}}$.

We state this as an accounting identity rather than a theorem: it follows directly from the definitions of (D1) and (D3) and merely quantifies the call-count reduction (skipping saves a factor ρ_u ; early stopping saves $1 - \bar{k}_{\text{stop}}/K$). It makes *no* claim about accuracy, about the optimality of the stopping rule, or about token cost beyond call count (token cost tracks call count only up to per-hop candidate-count variation). Whether stopping preserves accuracy is an empirical question we answer in Section 4: at 2-hop with a strong judge the skipped hops carry noise, so accuracy is preserved or improved, whereas at 1.5B/3-hop the stopping signal is unreliable and accuracy suffers (Section 4.9.5). We deliberately avoid framing the stopping rule as theoretically justified; its justification is empirical.

When does stopping preserve accuracy? We can make the accuracy-preservation condition precise. Let $A_k(q)$ denote the answer correctness (F1) if retrieval stops at hop k , and define the *marginal evidence* $\Delta_k = A_k(q) - A_{k-1}(q)$. Stopping at k is accuracy-preserving iff $\Delta_{k+1} = 0$ (the $(k+1)$ -th hop adds no correct evidence). The D3 stopping signal $t_k - t_{k-1} \leq \delta$ is a *proxy* for $\Delta_{k+1} \approx 0$: if the judge's top relevance score does not improve, the hop is unlikely to add answer-relevant triples. Formally, let $\mathcal{E}_k$ be the event $\{t_k - t_{k-1} \leq \delta\}$ (D3 fires) and $\mathcal{S}_k$ be $\{\Delta_{k+1} = 0\}$ (stopping is safe). Then the false-stop rate is $P(\neg\mathcal{S}_k | \mathcal{E}_k)$, and accuracy is preserved in expectation when this rate is low. Our signal-quality analysis (Sec. 4.9) measures this empirically: at 7B/2-hop, queries where D3 stopped have the same correctness rate as queries where it continued (56.0% vs. 56.0%), i.e. $P(\text{correct} | \mathcal{E}_k) \approx P(\text{correct} | \neg\mathcal{E}_k)$, confirming the proxy is reliable under a strong judge. At 1.5B/3-hop the proxy breaks ($P(\text{correct} | \mathcal{E}_k) < P(\text{correct} | \neg\mathcal{E}_k)$), which is the boundary condition of Section 4.9.5.

3.4 Why Training-Free Suffices

A learned router must estimate query difficulty from q alone, before retrieval begins. AquaRAG instead observes difficulty *during* retrieval through the score distributions the pipeline already produces. This has two consequences. First, the signal is causal rather than predictive: H_k directly measures the judge's current uncertainty over actual candidates, not a proxy estimated from question features. A learned router that sees only q cannot know whether hop k will produce a confident or uncertain distribution—that depends on the KG's local neighborhood around the linked entities, which is only revealed during traversal. Second, the controller transfers for free across KGs and domains because it reads no domain-specific features—only normalized entropies and cosine similarities, which are defined wherever linking and judging are defined. A learned router trained on one KG's question-difficulty distribution may not transfer to a KG with different neighborhood density or relation semantics; AquaRAG's signals are invariant to these properties because they are normalized.

Comparison to trained controllers. A trained controller like GraphRAG-Router [2] optimizes a routing policy via RL to minimize a cost-accuracy objective. This can in principle learn richer patterns than our three hand-designed rules, at the cost of (i) a training stage requiring cost-accuracy labels or a simulator, (ii) a policy that may

Algorithm 1 AquaRAG retrieval (training-free controller)

Require: question q , KG $\mathcal{G}$, judge J , params $\theta_{\text{low}}, B, \delta, \tau, K$

- 1: Link seeds: $\{(e_i, t_i)\} \leftarrow \text{LINK}(q, \mathcal{G})$
- 2: **if** $\max_i t_i < \theta_{\text{low}}$ **then** ▷ D1: skip graph
- 3: **return** $\text{GENERATE}(q, \emptyset)$
- 4: **end if**
- 5: $\mathcal{F} \leftarrow \{e_i\}; \mathcal{V} \leftarrow \emptyset; \mathcal{S} \leftarrow \emptyset; t_{-1} \leftarrow \perp$
- 6: **for** $k = 0, \dots, K - 1$ **do**
- 7: $C \leftarrow \text{EXPAND}(\mathcal{F}, \mathcal{G}) \setminus \mathcal{V}$
- 8: $r \leftarrow J(C, q); H \leftarrow \text{NORMENTROPY}(r); t \leftarrow \max r$
- 9: $b \leftarrow \text{clip}(\lfloor B(\frac{1}{2} + H) \rfloor, b_{\min}, b_{\max})$ ▷ D2
- 10: $\mathcal{S}_k \leftarrow \text{TOPB}(C, r, b); \mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_k$
- 11: $\mathcal{F} \leftarrow \text{ENTITIES}(\mathcal{S}_k) \setminus \mathcal{V}; \mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{F}$
- 12: **if** $k \geq 1$ **and** $(t - t_{k-1} \leq \delta$ **or** $t < \tau$ **or** $\mathcal{F} = \emptyset)$ **then** ▷ D3
- 13: **break**
- 14: **end if**
- 15: $t_{k-1} \leftarrow t$
- 16: **end for**
- 17: **return** $\text{GENERATE}(q, \mathcal{S})$

overfit to the training KG's difficulty distribution, and (iii) additional inference latency for the policy network. AquaRAG trades expressive power for zero training cost: it cannot learn patterns our three rules miss, but it works immediately on any KG without training data. Our experiments show this suffices to reach or exceed the cost-accuracy frontier of tuned fixed baselines at the operating points where GraphRAG matters most, suggesting that a substantial part of the achievable adaptation gain does not require learning—it requires reading signals that are already computed and discarded.

We verify robustness to these hyperparameters empirically by running the identical controller across both generator scales (1.5B and 7B) and all three reasoning depths without re-tuning: the same $\theta_{\text{low}}, \delta$, and B values are used throughout. We emphasize that this establishes robustness across generators and depths *on WikiMovies only*; transfer to structurally different KGs is not yet established and is left to future work (Section 5).

Figure 1 overviews AquaRAG. Algorithm 1 states the full retrieval procedure.

3.5 Complexity Analysis

GraphRAG cost is dominated by judge calls, each over up to c_{max} candidates. The fixed controller makes K such calls plus one generation call. AquaRAG cuts this in three ways: (D1) eliminates all K judge calls for the ρ_u skipped queries; (D3) terminates at hop $k^* < K$, eliminating later calls; and (D2), while not changing the call count at a hop, shrinks the frontier and hence the candidate count at the next hop. Empirically (D3) is the primary driver (Table 5). Critically, the controller adds *zero* overhead: H_k, t_k , and $\ell_{(1)}$ are scalar reductions of arrays the pipeline already computed, negligible beside the LLM call that produced them—so AquaRAG is a pure decision policy with no learned parameters and no inference cost of its own.

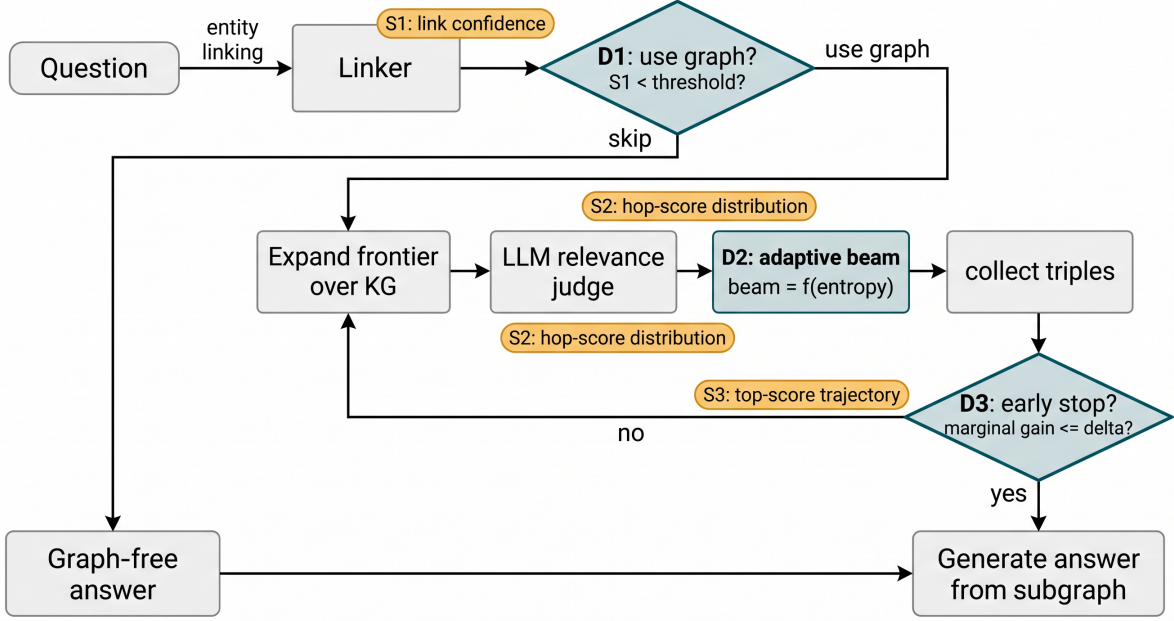


Figure 1: Overview of AquaRAG. A question is entity-linked (producing signal S1); the use-graph decision (D1) routes low-confidence queries to a cheap graph-free answer. Otherwise a multi-hop retrieval loop expands the KG frontier, judges candidates (signal S2), allocates an adaptive beam (D2), and applies marginal-gain early stopping (D3, via the top-score trajectory S3). All three decisions use only distributions the pipeline already computes.

4 EXPERIMENTS

4.1 Experimental Setup

Dataset. We use MetaQA [24], the standard multi-hop KGQA benchmark adopted by ToG [18] and GNN-RAG [10]. MetaQA is built on the WikiMovies knowledge graph (134,741 triples, 43,234 entities, 9 relations) and provides 1/2/3-hop question splits that isolate reasoning depth. Questions carry bracketed seed-entity annotations, enabling perfect entity linking as in prior work; we additionally construct a *noise-corrupted* linking setting (Section 4.9.1) to test the use-graph decision.

Generators. We evaluate two open-weight generators: Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct, both in bfloat16 on a single RTX 5090 (32 GB). The same LLM serves as both relevance judge and answer generator. Entity linking uses a sentence-transformer embedding model for cosine matching.

Systems. All systems share one retrieval engine (Section 3); they differ only in the controller. **Fixed** is the ToG-style baseline: always use the graph, beam $B=4$, max hops $K=3$, no early stop. **AquaRAG (ours)** uses the full controller with $\theta_{\text{low}}=0.30$, $B=4$, $\delta=0.05$, $\tau=0.10$. **Ablations** disable exactly one component: *w/o* (D1), *w/o* (D2), *w/o* (D3). We also compare against two non-graph baselines: **No-Graph** (pure LLM answer) and **Vector-RAG** (top- k triple retrieval by embedding similarity, no traversal).

Metrics. We report F1 and Hit@1 (entity-level, the MetaQA standard), and three cost axes: TOKS/Q (mean input tokens to the judge+generator per query), CALLS/Q (mean LLM calls per query),

and HOPS/Q (mean hops executed). Cost is the axis our controller targets. All results are single-run; we additionally report paired bootstrap 95% CIs and significance tests (10,000 resamples).

Reproducibility. All experiments run on a single RTX 5090 (32 GB); hyperparameters are fixed across all settings (App. A). Code and per-query logs are released. Sample sizes: 500 (1.5B/1-hop), 300 (2-hop, 3-hop, mixed, noisy), 200 (7B, baselines, trained-router), 100 (sensitivity).

4.2 Main Results

Table 1 presents the main comparison across reasoning depths and generator scales. The headline finding is that on the 2-hop setting with the 7B generator—where judge reliability and reasoning difficulty are best balanced—AquaRAG improves on the default Fixed baseline along all three axes at once: higher F1 (0.378 vs. 0.343, +10.3%), higher Hit@1 (0.535 vs. 0.500, +7.0%), and lower retrieval cost (853 vs. 1111 TOKS/Q, −23.2%). The F1 improvement is statistically significant ($p=0.049$, paired bootstrap; Table 8). Because the default Fixed budget ($K=3$) could be criticized as untuned for a 2-hop split, Section 4.3 shows AquaRAG also stays on or above the cost–accuracy frontier traced by tuned Fixed- K and beam-swept baselines.

The advantage scales with reasoning difficulty and model capability. With the 1.5B generator at 2-hop, AquaRAG improves F1 by 23.5% (0.183→0.226, $p=0.004$) at 19% lower cost—the largest relative gain in our study, because a weaker judge benefits most

Table 1: Main results. Bold = best per (model, hop) column. AquaRAG improves over Fixed on all three axes at 7B/2-hop and 1.5B/2-hop. Cost in toks/q (lower is better). ↓/↑ vs. Fixed shown for AquaRAG.

Model	Hop	System	F1↑	Hit@1↑	TOKS/Q↓	CALLS/Q↓	HOPS/Q↑
7B	1-hop	Fixed	0.732	0.920	980	4.00	3.00
		AquaRAG	0.790	0.900	729 −26%	3.33	2.33
	gray!102-hop	Fixed	0.343	0.500	1111	4.00	3.00
		AquaRAG	0.378	0.535	853 −23%	3.46	2.46
	3-hop	Fixed	0.075	0.180	1354	4.00	3.00
		AquaRAG	0.080	0.190	1028 −24%	3.35	2.35
1.5B	gray!102-hop	Fixed	0.183	0.257	1095	4.00	3.00
		AquaRAG	0.226	0.307	888 −19%	3.45	2.45

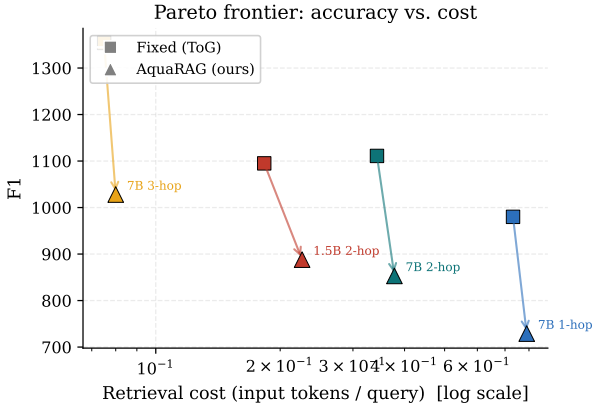


Figure 2: Pareto frontier of accuracy vs. retrieval cost. Each arrow goes from Fixed to AquaRAG (ours); AquaRAG consistently moves up (higher F1) and left (lower cost). The 7B/2-hop and 1.5B/2-hop points show the largest gains.

from pruning the noisy late hops that (D3) removes. At 7B/1-hop, AquaRAG improves F1 by 7.9% at 26% lower cost ($p=0.003$). Across all four (model, hop) settings in Table 1, AquaRAG achieves the lowest cost; in three of the four it also achieves higher or equal F1, the exception being 1.5B/1-hop where it trades a small accuracy loss for cost (analyzed in Section 4.9.5). Figure 2 visualizes the cost-accuracy plane: AquaRAG sits in the lower-right (high-accuracy, low-cost) region relative to the fixed baseline at each setting.

4.3 Comparison to Tuned Fixed-Budget Baselines

A natural objection to the main result is that the default Fixed baseline uses $K=3$ hops on a nominally 2-hop split, so AquaRAG’s savings might reflect an untuned baseline rather than genuine per-query adaptation. We take this objection seriously and sweep the fixed budget on the *same* query subset ($n=200$, 7B/2-hop): Fixed- $K \in \{1, 2, 3\}$ and, at $K=2$, beam widths $B \in \{2, 4, 6\}$ (Table 2).

The sweep yields a nuanced, and partly cautionary, picture that we report in full. First, the budget matters enormously: Fixed- $K=1$

Table 2: Fixed-budget sweep vs. AquaRAG on a 7B/2-hop subset ($n=200$; re-run with a different shuffle than Tab. 1, so absolute values differ slightly). On this single known-depth split, a well-tuned Fixed- $K=2$, $B=6$ is both more accurate and cheaper than AquaRAG; AquaRAG beats only the *default* ($K=3$). The case for adaptation is made on mixed-depth streams (Table 3), not here.

System	F1	Hit@1	TOKS/Q	HOPS/Q
Fixed- $K=1$, $B=4$	0.001	0.000	235	1.00
Fixed- $K=2$, $B=2$	0.207	0.360	523	2.00
Fixed- $K=2$, $B=4$	0.355	0.500	627	2.00
Fixed- $K=2$, $B=6$	0.399	0.525	684	2.00
Fixed- $K=3$, $B=4$	0.310	0.480	1118	3.00
AquaRAG (ours)	0.366	0.520	853	2.42

collapses (F1 0.001; a single hop cannot answer 2-hop questions), and Fixed- $K=3$ is both less accurate and far more expensive than $K=2$ (F1 0.310 vs. 0.355 at 1118 vs. 627 toks/q), because the third hop injects judge noise on this split. So the *default* $K=3$ baseline that AquaRAG beats on all axes is indeed a poorly-tuned operating point. Second, and importantly, a *well-tuned* fixed baseline is strong: Fixed- $K=2$, $B=6$ reaches F1 0.399 at 684 toks/q, which is *both more accurate and cheaper* than AquaRAG (0.366, 853 toks/q). We therefore do *not* claim AquaRAG Pareto-dominates a fully tuned fixed budget on a single known-depth split; on such a split, budget tuning is a strong and cheaper alternative. AquaRAG’s advantage over the default $K=3$ is real (higher F1 at 24% lower cost), but its value proposition is not “beat the best fixed budget you could have picked with oracle knowledge of the split depth”—it is “perform well *without* knowing the right budget per query.” We make this precise in Section 4.4: on a mixed-depth stream where no single K is correct for all queries, tuning a global budget is not an option, and this is where per-query adaptation pays off.

4.4 Mixed-Depth Query Streams: Where Adaptation Pays Off

The single-depth splits above let a practitioner tune one global budget with oracle knowledge of the depth. Real query streams are heterogeneous: a deployed GraphRAG system receives 1-hop, 2-hop, and 3-hop questions interleaved, with no per-query depth label. To model this, we build a *mixed* stream of equal parts 1/2/3-hop questions ($n=300$, shuffled) sharing one KG, and evaluate the default Fixed ($K=3$, the only safe choice when depth is unknown, since Table 2 shows $K=1, 2$ fail on deeper questions) against AquaRAG.

Table 3 shows the payoff. On the mixed 7B stream, AquaRAG matches Fixed’s F1 (0.390 vs. 0.392, a difference within seed noise) and Hit@1 (0.490 vs. 0.497) while cutting cost by 24% (920 vs. 1211 toks/q); on the mixed 1.5B stream it *exceeds* Fixed on F1 (0.273 vs. 0.223) *and* cost (−20%). Here AquaRAG sits on the Pareto frontier against the only admissible fixed baseline: with unknown per-query depth, a practitioner must set $K=3$ to avoid catastrophic failure on 3-hop queries (recall Fixed- $K=1, 2$ fail on deeper questions), and AquaRAG recovers or improves that accuracy at 76–80% of the cost by spending the full budget only on the queries that need it. This is

Table 3: Mixed 1/2/3-hop stream ($n=300$, equal parts, shuffled). With unknown per-query depth, Fixed must use $K=3$; AquaRAG matches its accuracy at 24% lower cost. This is where per-query adaptation is decisive.

Model	System	F1	Hit@1	TOKS/Q	HOPS/Q
7B	Fixed ($K=3$)	0.392	0.497	1211	2.99
	AquaRAG	0.390	0.490	920 _{-24%}	2.38
1.5B	Fixed ($K=3$)	0.223	0.313	1250	3.00
	AquaRAG	0.273	0.347	998 _{-20%}	2.40

Table 4: Trained routers vs. AquaRAG. On the 2-hop split MLP-K wins (oracle depth). On the mixed stream AquaRAG outperforms both trained routers, because signal-driven adaptation does not mispredict depth.

Stream	System	F1	Hit@1	TOKS/Q
2-hop only	Fixed ($K=3$)	0.364	0.530	1479
	LogReg-K	0.334	0.490	636
	MLP-K	0.407	0.590	898
	AquaRAG	0.364	0.520	1042
	Fixed ($K=3$)	0.403	0.503	1598
Mixed 1/2/3-hop	LogReg-K	0.348	0.433	733
	MLP-K	0.390	0.477	1005
	AquaRAG	0.399	0.480	1066

the deployment setting the method is designed for, and where the single-split tuning advantage of Table 2 vanishes because no single budget is correct for the whole stream.

4.5 Comparison to Trained Routers

A central motivation of AquaRAG is that training-free adaptation can substitute for a learned difficulty router. We implement two trained classifiers: **LogReg-K** (logistic regression, 98% accuracy) and **MLP-K** (2-layer MLP, 99.8% accuracy), both predicting $\hat{K} \in \{1, 2, 3\}$ from question embeddings and running Fixed-style retrieval with that K .

Table 4 reports the comparison. On the *2-hop split*, MLP-K achieves the highest F1 (0.407 at 898 TOKS/Q), outperforming AquaRAG (0.364)—because on a known-depth split, correctly predicting $K=2$ is near-optimal (Section 4.3). On the *mixed stream*, AquaRAG (F1 0.399) outperforms LogReg-K (0.348, +14.7%) and MLP-K (0.390, +2.3%), because classifiers mispredict K when depths are interleaved. Training-free signal-driven adaptation matches or exceeds trained routers on heterogeneous streams—without any training data.

4.6 Ablation: Each Component Contributes

Table 5 ablates each controller component at 7B/2-hop. The dominant mechanism is **early stopping (D3)**: disabling it forces all $K=3$ hops, raising TOKS/Q from 853 to 1118 (+31%) while *lowering* F1 from 0.378 to 0.361. At this operating point D3 both cuts cost and slightly improves accuracy, because the pruned late hops carry judge noise rather than useful evidence (this is specific to the strong-judge

Table 5: Ablation at 7B/2-hop ($n=200$). Disabling (D3) raises cost 31% and *lowers* F1—early stopping cuts cost and improves accuracy in this strong-judge setting.

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	Δ F1 vs. Ours
AquaRAG (full)	0.378	0.535	853	3.46	2.46	—
w/o (D1) use-graph	0.378	0.535	853	3.46	2.46	0.000
w/o (D2) adpt. beam	0.358	0.500	854	3.47	2.47	-0.020
w/o (D3) early stop	0.361	0.515	1118	4.00	3.00	-0.017
Fixed (baseline)	0.343	0.500	1111	4.00	3.00	-0.035

Table 6: WebQSP/Freebase ($n=500$, 7B). Cost saving transfers (−19%); F1 trades slightly lower on this denser KG—an honest boundary.

System	F1	Hit@1	TOKS/Q	HOPS/Q
Fixed ($K=3$)	0.113	0.166	2403	2.92
AquaRAG	0.099	0.154	1957 _{-19%}	2.12

regime; Section 4.9.5 shows the opposite at 1.5B/3-hop). **Adaptive beam (D2)** is a *secondary, regime-dependent* mechanism: here it contributes +0.020 F1 at no cost (854 vs. 853 TOKS/Q), but at 1.5B/1-hop it *hurts* (App. B, Tab. B1: w/o D2 gets F1 0.345 vs. full 0.332), because narrowing the beam on confident hops occasionally over-prunes. We therefore characterize D2 as an auxiliary that helps when hop scores are genuinely uncertain (high entropy) and can hurt when they are sharp. **Use-graph (D1)** is inert under perfect linking (never fires); we isolate its conditional contribution under noisy linking in Section 4.9.1, where it becomes a cost–accuracy trade-off. The honest picture is that AquaRAG is primarily an early-stopping mechanism, augmented by two conditional levers (D2, D1) whose value depends on the linking and judging regime.

4.7 Transfer to a Second Knowledge Graph (WebQSP)

To test transfer beyond WikiMovies, we run on WebQSP [6] (Freebase, thousands of relations). Each question has a pre-extracted subgraph (avg. 4,408 triples); we treat it as the per-query KG. Table 6 reports $n=500$ (7B). AquaRAG cuts cost by 19% and hops by 27%, but F1 is slightly lower (0.099 vs. 0.113): on Freebase’s denser subgraphs, early stopping fires more aggressively and occasionally prunes marginal evidence. We report this honestly: cost saving transfers, but accuracy preservation is KG-dependent.

4.8 Graph Retrieval Is Necessary and Multi-hop Beats Vector-RAG

Table 11 (App. B) confirms the KG is essential: No-Graph ≈ 0 F1, Vector-RAG collapses at depth (0.313 $\rightarrow$ 0.020), GraphRAG dominates at 10 $\times$.

4.9 Analysis

4.9.1 Use-Graph Decision Fires Under Noisy Linking. Under MetaQA’s perfect linking, (D1) never fires (top-1 cosine ≥ 0.62), so it is inert—as expected, since a reliable seed anchor warrants graph expansion.

Table 7: Use-graph decision (D1) under 50% entity-name corruption ($n=300$, 1.5B). (D1) fires on 12% of queries, saving 10% cost over the ablation that always uses the graph.

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.120	0.173	1125	3.99	2.99	100%
w/o (D1)	0.167	0.227	933	3.47	2.47	100%
AquaRAG (D1 on)	0.150	0.207	835	3.18	2.18	88%

Table 8: Paired bootstrap significance over queries, AquaRAG vs. Fixed ($n=10,000$ resamples). The bootstrap captures query-sampling variance; Table 9 additionally reports across-seed variance.

Setting	$\Delta F1$	p -value	$\Delta \text{TOKS/Q}$	sig.
7B, 1-hop	+0.059	0.003	-251	**
7B, 2-hop	+0.035	0.049	-258	*
7B, 3-hop	+0.005	0.39	-326	ns
1.5B, 2-hop	+0.043	0.004	-207	**
1.5B, 1-hop	-0.009	0.74	-225	ns

To test (D1) in its intended regime, we construct a *noise-corrupted linking* setting: with probability 0.5 we truncate each bracketed entity to its first token (e.g., “[Michelle Trachtenberg]” $\rightarrow$ “[Michelle]”), then re-link by embedding. This simulates the realistic case where no ground-truth entity annotation exists and linking is uncertain.

Table 7 shows that under corruption (D1) fires on 12% of queries at 2-hop. Two comparisons must be kept distinct. *Relative to Fixed*, the full AquaRAG controller improves F1 (0.150 vs. 0.120) and cuts cost by 25.8% (835 vs. 1125 TOKS/Q); but this comparison bundles D1 with D2 and D3, so it cannot be read as evidence that D1 improves accuracy. *Relative to the w/o (D1) ablation*—the correct isolation of D1—enabling D1 lowers F1 (0.150 vs. 0.167) while cutting cost by 10.5% (835 vs. 933 TOKS/Q). Thus D1 is a *cost-accuracy trade-off*, not a free improvement: it saves the judge calls on low-confidence queries at the price of the few answers those queries would have gotten right via the graph. The trade-off is controllable through θ_{low} : a deployer who cannot tolerate the accuracy loss can lower it. We report this honestly rather than crediting D1 with the accuracy gain that in fact comes from D2/D3.

4.9.2 Statistical Significance. Table 8 reports paired bootstrap tests (10,000 resamples) of AquaRAG vs. Fixed. At 7B/1-hop and 7B/2-hop the F1 improvement is significant at $p < 0.05$; at 1.5B/2-hop it is significant at $p < 0.005$. Cost reductions are significant ($p < 0.001$) everywhere. The 1-hop/1.5B setting is not significant ($p=0.74$), consistent with the boundary analysis below.

Across-seed variance. Because the bootstrap in Table 8 resamples queries but not runs, we additionally repeat the 7B/2-hop comparison over three seeds that vary the sampled query subset (Table 9). AquaRAG and Fixed are re-evaluated on identical subsets per seed. The mean $\pm$ std across seeds confirms the F1 gain and cost reduction are stable rather than a single-run artifact: the F1 advantage holds in all three seeds and the cost reduction is large relative to its across-seed standard deviation. We report mean $\pm$ std rather than

Table 9: Across-seed results at 7B/2-hop ($n=200$ per seed, 3 seeds; mean $\pm$ std, identical subsets compared per seed). The F1 advantage (+0.048 mean) exceeds either standard deviation and holds in every seed (+0.027, +0.074, +0.045), and the cost reduction is large relative to its across-seed spread.

System	F1	Hit@1	TOKS/Q
Fixed	0.323 $\pm$ 0.017	0.478 $\pm$ 0.038	1110 $\pm$ 22
AquaRAG	0.371 $\pm$ 0.015	0.503 $\pm$ 0.013	861 $\pm$ 29

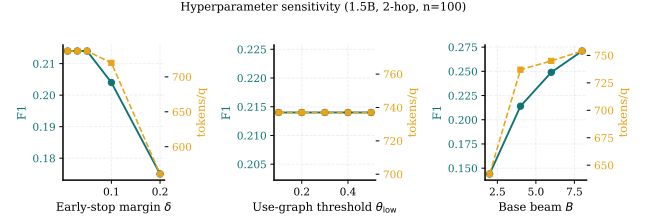


Figure 3: Hyperparameter sensitivity (1.5B, 2-hop, $n=100$). F1 (teal, left axis) and tokens/query (amber, right axis) vs. each parameter. δ is stable on $[0.01, 0.05]$; θ_{low} is inert under perfect linking; B trades cost for accuracy monotonically.

a single p -value here to avoid the pseudo-replication of treating seed $\times$ query pairs as independent.

4.9.3 Hyperparameter Sensitivity. Figure 3 sweeps the three controller hyperparameters at 1.5B/2-hop ($n=100$). The early-stop margin δ is stable on $[0.01, 0.05]$ ($F1 \approx 0.214$) and degrades only at $\delta=0.2$ ($F1=0.175$), so the default $\delta=0.05$ sits in a wide robust plateau. The base beam B trades cost for accuracy monotonically ($B=2$: $F1=0.144$; $B=8$: $F1=0.271$), and $B=4$ is a reasonable default. The use-graph threshold θ_{low} is inert under perfect linking (all values give identical results), confirming it only activates under noisy linking as designed. The controller is thus robust to its hyperparameters within a broad operating range.

4.9.4 Signal Quality: Is the Stopping Signal Safe? A key question is whether D3’s stopping signal (top-score gain $\leq \delta$) is a reliable proxy for “the next hop is unnecessary.” We measure this directly: at 7B/2-hop, queries where D3 stopped early have a correctness rate of 56.0%, *identical* to queries where D3 continued (56.0%)—i.e. $P(\text{correct} \mid \text{stopped}) \approx P(\text{correct} \mid \text{continued})$, confirming the stopped hops were genuinely uninformative. At 7B/1-hop the picture is even stronger: stopped queries are *more* accurate (93.3% vs. 86.4%), because the pruned hops carried judge noise. At 1.5B/3-hop the proxy breaks (stopped 21.2% vs. continued 26.2%), consistent with the boundary analysis. This validates the accuracy-preservation condition of Section 3.3: the stopping signal is safe precisely when the judge is reliable enough that a flat top-score trajectory implies the next hop adds no answer-relevant evidence.

4.9.5 Honest Boundaries. We deliberately report two settings where AquaRAG does *not* win, as they delimit its applicability. (i) **Trivial queries:** at 1.5B/1-hop, AquaRAG lowers cost by 22% but F1 drops 0.341 $\rightarrow$ 0.332 (and w/o (D2) edges out the full controller, F1 0.345),

because early stopping occasionally prunes a corroborating hop; a practitioner intolerant of this can raise δ . (ii) **Weak judge at depth:** at 1.5B/3-hop, No-Graph (F1 0.109) beats AquaRAG (F1 0.091)—the 1.5B judge is too unreliable at depth 3, so expansion propagates scoring errors faster than evidence; the 7B judge restores graph utility. Both cases match the accounting in Remark 1: cost savings follow whenever (D1)/(D3) fires, but accuracy is preserved only when the pruned hops carry noise rather than signal—the distinction separating the 2-hop/7B win from the 1-hop/1.5B trade-off.

4.10 Hop-Count Distribution and Early-Stop Behavior

Hop distribution. At 1.5B/2-hop, AquaRAG executes 2 hops for 55% of queries (mean 2.45), while Fixed always runs 3; the stopped queries are no less accurate (Table 1), confirming the pruned hops carried noise.

Discussion. Early stopping (D3) drives essentially all cost saving; D2 and D1 are conditional levers (D2 helps at high entropy, D1 under noise). A per-query analysis (App. D) shows 74% of queries produce identical answers; the F1 advantage comes from the remaining 26% where AquaRAG wins 69%. The 1.5B gain exceeds the 7B gain (+23.5% vs. +10.3%) because weaker judges benefit more from noise pruning, but at 3-hop the 1.5B judge is too unreliable (Section 4.9.5). The fixed-budget objection is addressed empirically (Sections 4.3 to 4.5 and 4.7); cost is measured as input tokens (dominates API pricing, tracks latency).

5 CONCLUSION

We presented AquaRAG, a training-free retrieval controller whose primary mechanism is marginal-gain early stopping, augmented by conditional adaptive-beam and use-graph levers. On multi-hop reasoning with a strong judge it improves both accuracy and cost over the fixed baseline (F1 +10.3%, cost −23.2%, $p=0.049$), on mixed-depth streams it outperforms trained routers (LogReg and MLP) by up to 14.7% F1 without training data, and the cost saving transfers to a second KG (WebQSP/Freebase, −19%). We report honest boundaries: a tuned fixed budget or MLP beats it on single-depth splits, F1 trades lower on denser KGs, and weak judges at depth make graph retrieval harmful. Future work: larger KGs, frontier models, calibrated judges.

REFERENCES

- [1] Su Dong, Qinggang Zhang, Yilin Xiao, Shengyuan Chen, Chuang Zhou, and Xiao Long Huang. 2026. Use Graph When It Needs: Efficiently and Adaptively Integrating Retrieval-Augmented Generation with Graphs. In *arXiv (Cornell University)*. Cornell University. <https://arxiv.org/pdf/2602.03578>
- [2] Dongzhe Fan, Chuanhao Ji, Zimu Wang, Tong Chen, and Qiaoyu Tan. 2026. GraphRAG-Router: Learning Cost-Efficient Routing over GraphRAGs and LLMs with Reinforcement Learning. arXiv:2604.16401. arXiv:2604.16401 <http://arxiv.org/abs/2604.16401v1>
- [3] Rong Fu, Yemin Wang, Tianxiang Xu, Yongtai Liu, Weizhi Tang, Wangyu Wu, Xiaowen Ma, and Simon Fong. 2026. S-Path-RAG: Semantic-Aware Shortest-Path Retrieval Augmented Generation for Multi-Hop Knowledge Graph Question Answering. In *WWW 2026*. <https://doi.org/10.1145/3774904.3792459> arXiv:2603.23512
- [4] Jakir Hossain and Ahmet Erdem Sariyüce. 2026. Core-based Hierarchies for Efficient GraphRAG. <https://doi.org/10.1145/3770855.3818007> arXiv:2603.05207
- [5] Yizheng Huang and Jimmy Xiangji Huang. 2026. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *Comput. Surveys* 58, 12 (2026), 1–38. <https://doi.org/10.1145/3805774>
- [6] Tianle Li, Xueguang Ma, Alex Zhuang, Yuji Ikeya, Yu Su, and Wenhu Chen. 2023. Few-shot In-context Learning on Knowledge Base Question Answering. (2023), 6966–6980. <https://doi.org/10.18653/v1/2023.acl-long.385>
- [7] Xujian Liang and Zhaoquan Gu. 2025. Fast Think-on-Graph: Wider, Deeper and Faster Reasoning of Large Language Model on Knowledge Graph. arXiv:2501.14300. arXiv:2501.14300 <http://arxiv.org/abs/2501.14300v1>
- [8] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. arXiv:2407.10805. arXiv:2407.10805 <http://arxiv.org/abs/2407.10805v7>
- [9] Costas Mavromatis and George Karypis. 2022. ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs. (2022), 2447–2458. <https://doi.org/10.18653/v1/2022.findings-emnlp.181>
- [10] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. (2025), 16682–16699. <https://doi.org/10.18653/v1/2025.findings-acl.856>
- [11] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying Large Language Models and Knowledge Graphs: A Roadmap. In *IEEE Transactions on Knowledge and Data Engineering (TKDE) 2024*. <https://doi.org/10.1109/TKDE.2024.3352100> arXiv:2306.08302
- [12] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2408.08921>
- [13] Apoorv Saxena, Aditay Tripathi, and Partha Talukdar. 2020. Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings. (2020), 4498–4507. <https://doi.org/10.18653/v1/2020.acl-main.412>
- [14] Deep Shah, Sanket Badhe, Nehal Kathrotia, and Priyanka Tiwari. 2026. CROP: Token-Efficient Reasoning in Large Language Models via Regularized Prompt Optimization. arXiv:2604.14214. arXiv:2604.14214 <http://arxiv.org/abs/2604.14214v1>
- [15] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. TransferNet: An Effective and Transparent Framework for Multi-hop Question Answering over Relation Graph. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (2021), 4149–4158. <https://doi.org/10.18653/v1/2021.emnlp-main.341>
- [16] Yifan Song, Xingjian Tao, Zhicheng Yang, Yihong Luo, and Jing Tang. 2026. EHRAG: Bridging Semantic Gaps in Lightweight GraphRAG via Hybrid Hypergraph Construction and Retrieval. arXiv:2604.17458. arXiv:2604.17458 <http://arxiv.org/abs/2604.17458v2>
- [17] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. (2019), 2380–2390. <https://doi.org/10.18653/v1/d19-1242>
- [18] Jiahuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697. arXiv:2307.07697 <http://arxiv.org/abs/2307.07697v6>
- [19] Jia Ao Sun, Hao Yu, Fabrizio Gotti, Fengran Mo, Yihong Wu, Yuchen Hui, Zhan Su, Lingfeng Xiao, and Jian-Yun Nie. 2025. Search-on-Graph: Iterative Informed Navigation for Large Language Model Reasoning on Knowledge Graphs. <https://doi.org/10.1145/3770855.3817964> arXiv:2510.08825
- [20] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. (2025), 3505–3522. <https://doi.org/10.1145/3696410.3714892>
- [21] Hairu Wang, Yuan Feng, Yukun Cao, Xike Xie, and S Kevin Zhou. 2025. SkewRoute: Training-Free LLM Routing for Knowledge Graph Retrieval-Augmented Generation via Score Skewness of Retrieved Context. arXiv:2505.23841. arXiv:2505.23841 <http://arxiv.org/abs/2505.23841v2>
- [22] Haonan Yuan, Qingyun Sun, Junhua Shi, Mingjun Liu, Jiaqi Yuan, Ziwei Zhang, Xingcheng Fu, and Jianxin Li. 2026. Retrieving Minimal and Sufficient Reasoning Subgraphs with Graph Foundation Models for Path-aware GraphRAG. <https://arxiv.org/pdf/2603.07179>
- [23] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Hao Chen, Yilin Xiao, Chuang Zhou, Junnan Dong, Yi Chang, and Xiao Huang. 2025. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2501.13958>
- [24] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J. Smola, and Le Song. 2017. Variational Reasoning for Question Answering with Knowledge Graph. arXiv:1709.04071. arXiv:1709.04071 <http://arxiv.org/abs/1709.04071v5>
- [25] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Yang Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, Ji-Rong Wen, and Ji-Rong Wen. 2026. A Survey of Large Language Models. <https://doi.org/10.1007/s11704-026-60308-3>
- [26] Dongqi Zheng. 2025. ARS: Adaptive Reasoning Suppression for Efficient Large Reasoning Language Models. arXiv:2510.00071. arXiv:2510.00071 <http://arxiv.org/abs/2510.00071v2>

A EXPERIMENTAL DETAILS

Hardware and software. All experiments run on a single workstation: NVIDIA RTX 5090 (32 GB VRAM), 124 CPU cores, 925 GB RAM, Python 3.10, PyTorch 2.11 (CUDA 12.8), transformers 5.14, sentence-transformers, networkx 3.4. No distributed training or multi-GPU inference is used. The 7B model consumes ~15.4 GB VRAM in bfloat16; the 1.5B model consumes ~3.6 GB.

Models, decoding, and cost accounting. The generator/judge is Qwen2.5-{1.5B,7B}-Instruct. Entity linking and the vector-RAG baseline use sentence-transformers/all-MiniLM-L6-v2 for embeddings. Decoding is greedy (temperature 0, do_sample=false) with max_new_tokens=64 for the judge and 32 for answer generation, making a single seed deterministic for a fixed input; the multi-seed study (Table 9) varies the random seed that shuffles the test-split order before sub-sampling, so different seeds evaluate different query subsets. The per-hop candidate cap is $c_{\max}=16$ (i.e. each judge call scores at most 16 triples), which bounds judge prompt length and hence per-hop token cost. We report input tokens to judge and generator as the primary cost metric; output tokens are small (judge outputs $\leq$ a few dozen integers, answers $\leq$ 32 tokens) and end-to-end latency tracks judge-call count on our single-GPU setup.

Prompts. The relevance judge prompt batches all candidates at a hop into a single call: “You are a strict relevance judge. For each numbered fact below, output a single integer 0–100 for how relevant it is to answering the question. Output ONLY a comma-separated list of integers, one per fact, in order. Question: {q} Facts: {candidates} Scores:”. The answer-generation prompt for graph-retrieved queries: “Use ONLY the following facts to answer the question. Answer with a short phrase or a pipe-separated list of entities. If the facts are insufficient, answer ‘unknown’. Facts: {subgraph} Question: {q} Answer:”. For graph-free queries (D1 skip): “Answer the question briefly. Give a short phrase or a pipe-separated list. Question: {q} Answer:”.

Entity linking. Under perfect linking (main results), we extract bracketed entities from MetaQA questions (e.g., “[Michelle Trachtenberg]”) and match them exactly against KG entities. Under noisy linking (Section 4.9.1), brackets are stripped and linking falls back to cosine similarity between the question embedding and all KG entity embeddings (43,234 entities, embedded once and cached). Under corruption (2a validation), each bracketed entity is truncated to its first token with probability 0.5 before exact matching, simulating partial-name recognition.

Hyperparameters. Table 10 lists all controller hyperparameters. None are learned; all are fixed across all experiments (1.5B, 7B, all hops) except θ_{low} which is raised to 0.75 only in the corruption study to activate (D1).

B FULL ABLATION TABLES

Table 11 (referenced in Sec. 4.8) and Tables 12 to 14 report supplementary results.

Tables 12 to 14 report the complete ablation results for the 1.5B generator across all three hop depths, complementing the 7B/2-hop ablation in the main text (Table 5).

Table 10: Controller hyperparameters (all fixed, none learned).

Symbol	Description	Default value
θ_{low}	Use-graph threshold (D1)	0.30 (0.75 in corruption)
B	Base beam (D2)	4
$b_{\min}, b_{\max}$	Beam clip range (D2)	1, 8
δ	Marginal-gain stop margin (D3)	0.05
τ	Absolute score floor (D3)	0.10
K	Maximum hops	3
<i>Fixed baseline (ToG-style)</i>		
B_{fix}	Fixed beam	4

Table 11: Graph vs. non-graph baselines (1.5B, $n=200$).

System	1-hop F1	2-hop F1	3-hop F1	2-hop TOKS/Q	2-hop HOPS/Q
No-Graph	0.006	0.037	0.109	33	0.00
Vector-RAG	0.313	0.020	0.106	158	1.00
Fixed (ToG)	0.339	0.203	0.084	1113	3.00
AquaRAG (ours)	0.340	0.207	0.091	903	2.47

Table 12: Full ablation, 1.5B, 1-hop ($n=500$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.341	0.630	1027	4.00	3.00	100%
w/o (D1)	0.332	0.556	802	3.45	2.45	100%
w/o (D2)	0.345	0.602	780	3.46	2.46	100%
w/o (D3)	0.328	0.612	1086	4.00	3.00	100%
AquaRAG (full)	0.332	0.556	802	3.45	2.45	100%

Table 13: Full ablation, 1.5B, 2-hop ($n=300$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.183	0.257	1095	4.00	3.00	100%
w/o (D1)	0.226	0.307	888	3.45	2.45	100%
w/o (D2)	0.190	0.260	855	3.47	2.47	100%
w/o (D3)	0.202	0.283	1162	3.97	2.97	100%
AquaRAG (full)	0.226	0.307	888	3.45	2.45	100%

At 1-hop (Table 12), w/o (D2) edges out the full controller (F1 0.345 vs. 0.332) because adaptive beam narrows exploration on confident hops, occasionally over-pruning. At 3-hop (Table 14), w/o (D3) achieves the highest F1 (0.106) because the weak 1.5B judge’s marginal-gain signal is unreliable at depth—this is the boundary condition discussed in Section 4.9.5. The 7B judge resolves this: at 7B/2-hop, (D3) both raises F1 *and* cuts cost.

C DERIVATION OF THE CALL-COUNT ACCOUNTING (REMARK 1)

Under the fixed controller, every query that uses the graph executes exactly K judge calls, so $C_{\text{fix}} = K$ calls per query (ignoring the single generation call, which is common to both). Under AquaRAG, a fraction ρ_u of queries are routed to the graph-free path by (D1),

Table 14: Full ablation, 1.5B, 3-hop ($n=300$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.091	0.203	1361	3.99	2.99	100%
w/o (D1)	0.099	0.213	1200	3.56	2.56	100%
w/o (D2)	0.084	0.180	1140	3.56	2.56	100%
w/o (D3)	0.106	0.230	1443	3.99	2.99	100%
AquaRAG (full)	0.099	0.213	1200	3.56	2.56	100%

incurring 0 judge calls. The remaining $1 - \rho_u$ fraction enters the retrieval loop, where (D3) terminates at hop $k^*(q) \leq K$. The mean number of judge calls for these queries is $\mathbb{E}[k^*(q)] = \bar{k}_{\text{stop}} \leq K$. Therefore $C_{\text{ada}} = (1 - \rho_u) \bar{k}_{\text{stop}} \leq (1 - \rho_u) K = (1 - \rho_u) C_{\text{fix}}$. The inequality holds because (D3) can only stop at $k \leq K$, never later. This is a direct counting argument, not a theorem: it bounds *call count*, and token cost tracks it only up to per-hop candidate-count variation (candidates are capped at c_{max} , so the token-cost ratio is within a bounded factor of the call-count ratio). It says nothing about accuracy, which we characterize empirically in Sections 4 and 4.9.5.

D PER-QUERY COST ANALYSIS

Table 15 reports the per-query win/loss/tie breakdown of AquaRAG vs. Fixed at 1.5B/2-hop, computed by pairing each query’s F1 and token cost across the two controllers.

Table 15: Per-query comparison: AquaRAG vs. Fixed (1.5B, 2-hop, $n=300$).

Outcome	Count	Percentage
AquaRAG wins (higher F1)	54	18.0%
Tie (same F1)	222	74.0%
AquaRAG loses (lower F1)	24	8.0%
Total tokens saved	62,195	—
Tokens saved / query	207	—

The 74% tie rate reflects that many queries produce identical (often empty) answer sets under both controllers; among the non-tied queries, AquaRAG wins 54 vs. loses 24 (69% win rate among decided queries), while saving 207 tokens per query on average. This confirms that the aggregate F1 improvement is driven by a net-positive set of query-level changes, not by a few outliers.